



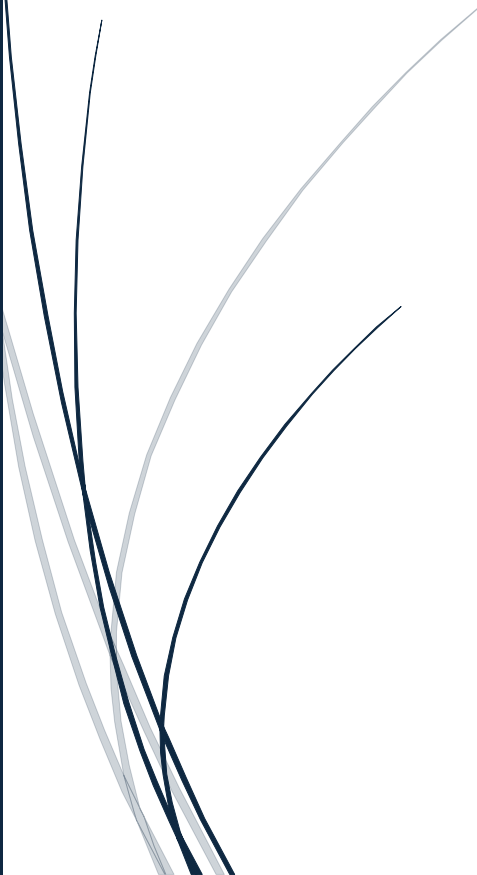
18 de septiembre de 2026

LAB: FILE PATH TRAVERSAL SIMPLE CASE

CNO IV – Seguridad Informática

Profesor: Servando López Contreras
Universidad Politécnica de San Luis Potosí

Luis Felipe Jaramillo Barajas
MATRICULA: 183559



Contenido

1. Introducción	2
2. Objetivos	2
Objetivo general.....	2
Objetivos específicos	2
3. Preparación del laboratorio	3
4. Fase 1: Acceso a la aplicación objetivo	5
5. Fase 2: Interceptación de la petición de imagen	6
6. Fase 3: Confirmación del parámetro vulnerable	8
7. Fase 4: Explotación: construcción del payload.....	8
7.1 Explicación del payload.....	10
7.2 Resolución del laboratorio	10
8. Mitigación recomendada.....	11
9. Conclusiones	11

1. Introducción

Este informe documenta el proceso completo de una práctica de laboratorio especializado en seguridad de aplicaciones web, realizada sobre la plataforma PortSwigger Web Security Academy, específicamente el laboratorio “File path traversal, simple case”. El objetivo de este ejercicio fue identificar y explotar una vulnerabilidad de path traversal (traversal de directorios) presente en la funcionalidad de visualización de imágenes de productos de una tienda en línea de prueba, con el fin de acceder a un archivo sensible del sistema operativo subyacente, el archivo `/etc/passwd`.

Todo el ejercicio se realizó dentro del entorno controlado que proporciona la propia plataforma de PortSwigger, utilizando Burp Suite Community Edition como proxy de interceptación para analizar y modificar las peticiones HTTP entre el navegador y el servidor de la aplicación vulnerable.

2. Objetivos

Objetivo general

Demostrar la capacidad de identificar, analizar y explotar una vulnerabilidad de path traversal en una aplicación web, aplicando herramientas de interceptación de tráfico HTTP y comprendiendo los fundamentos teóricos que sustentan este tipo de ataque.

Objetivos específicos

- Configurar Burp Suite Community Edition como proxy de interceptación entre el navegador y la aplicación objetivo.
- Identificar el parámetro de la aplicación susceptible a manipulación de rutas de archivo.
- Interceptar y modificar una petición HTTP legítima para construir un payload de path traversal.
- Acceder al contenido del archivo `/etc/passwd` fuera del directorio previsto por la aplicación.
- Comprender los fundamentos teóricos de la vulnerabilidad y su clasificación.
- Proponer medidas de mitigación adecuadas para prevenir este tipo de vulnerabilidad.

3. Preparación del laboratorio

El laboratorio utilizado, “File path traversal, simple case”, es proporcionado por PortSwigger a través de su plataforma Web Security Academy. Se trata de un entorno de nivel Apprentice que contiene una vulnerabilidad de path traversal en la funcionalidad que muestra las imágenes de los productos de una tienda en línea ficticia (“We Like To Shop”). El objetivo declarado del laboratorio es recuperar el contenido del archivo `/etc/passwd`.



Figura 1. Descripción del laboratorio “File path traversal, simple case” en PortSwigger Web Security Academy.

Como herramienta de interceptación se utilizó Burp Suite Community Edition, iniciando un proyecto temporal en memoria y cargando la configuración por defecto de la herramienta.

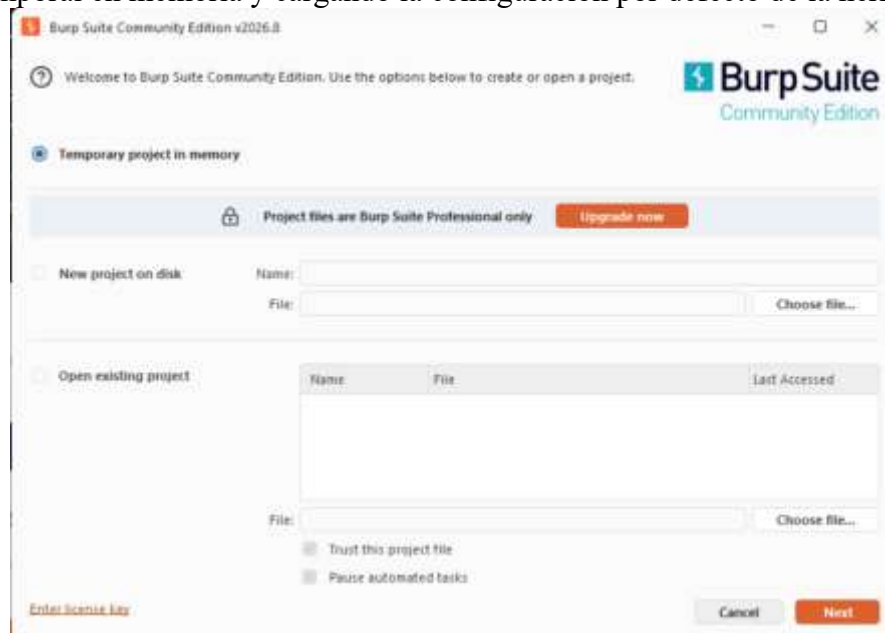


Figura 2. Pantalla de bienvenida de Burp Suite Community Edition — selección de proyecto temporal en memoria.



Figura 3. Selección de la configuración por defecto de Burp Suite para iniciar el proyecto.

Concepto teórico: Burp Suite como proxy de interceptación. Burp Suite es una plataforma para realizar pruebas de seguridad en aplicaciones web. Su componente Proxy actúa como un intermediario (man-in-the-middle) entre el navegador y el servidor, permitiendo interceptar, inspeccionar y modificar las peticiones y respuestas HTTP/HTTPS antes de que lleguen a su destino. Esto es fundamental para pruebas de path traversal, ya que permite alterar manualmente parámetros de la petición (como nombres de archivo) que en el flujo normal de la aplicación no serían editables desde la interfaz web.

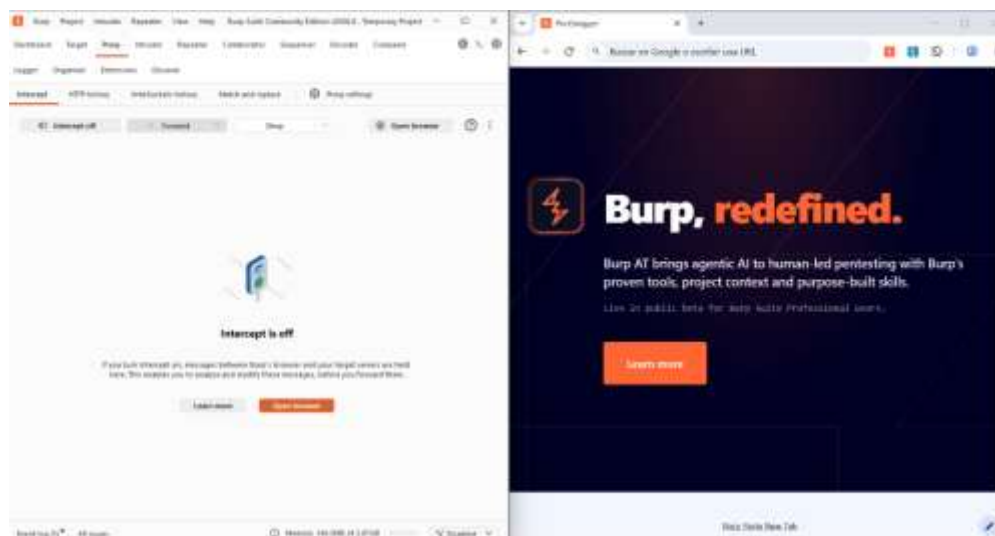


Figura 4. Navegador integrado de Burp Suite abierto junto al proxy, con la interceptación inicialmente desactivada.

4. Fase 1: Acceso a la aplicación objetivo

Utilizando el navegador integrado de Burp Suite, se accedió a la URL generada por PortSwigger para esta instancia del laboratorio, correspondiente a la tienda en línea “We Like To Shop”. Esta aplicación de prueba muestra un catálogo de productos.

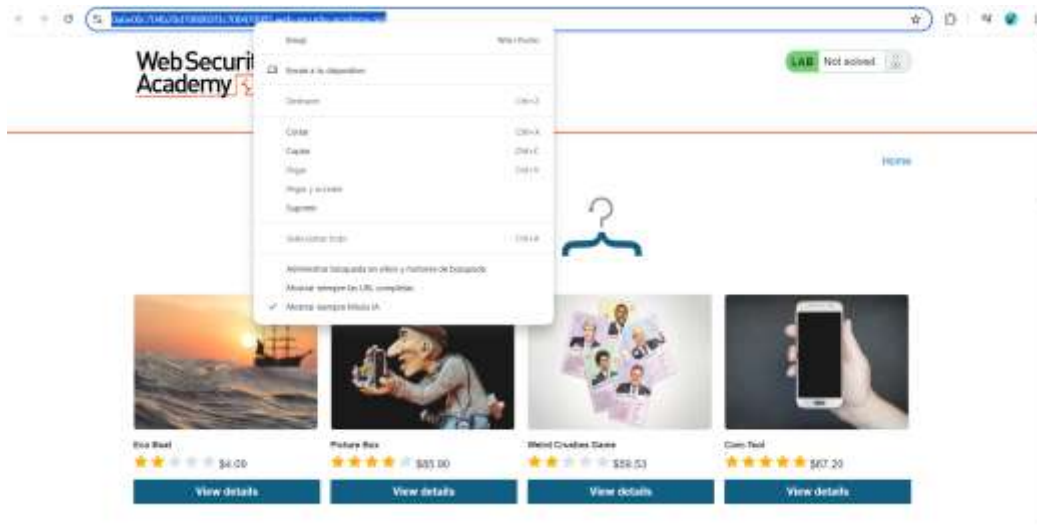


Figura 5. URL única del laboratorio asignada por PortSwigger, visible en la barra de direcciones del navegador.

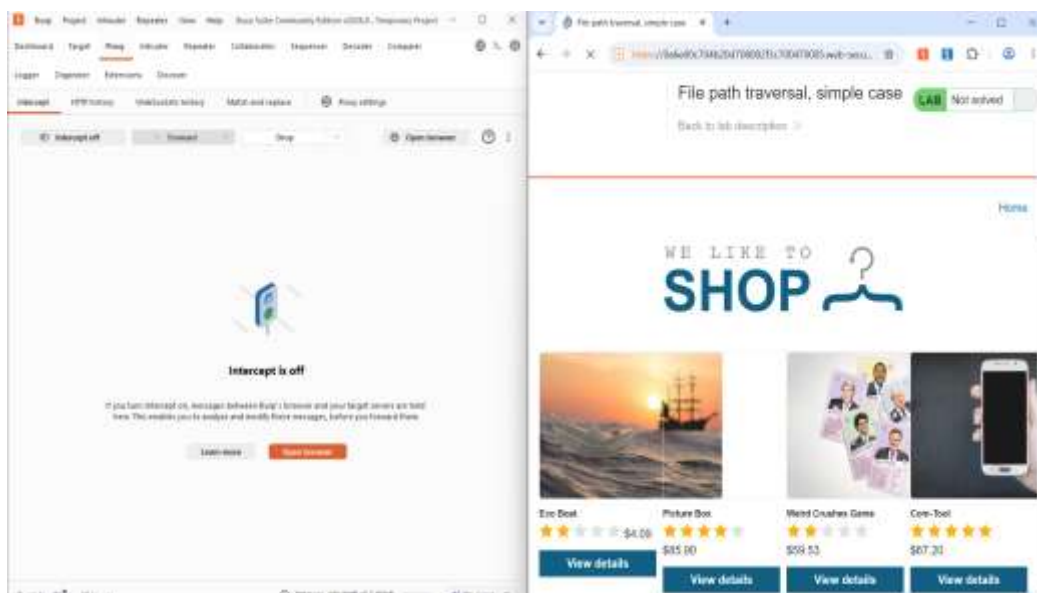


Figura 6. Página principal de la tienda “We Like To Shop” mostrando el catálogo de productos con sus imágenes.

Concepto teórico: reconocimiento de la superficie de ataque. Antes de intentar cualquier explotación, es necesario identificar los puntos de la aplicación que reciben datos controlados por el usuario y que interactúan con el sistema de archivos del servidor. En este caso, cada imagen de producto se carga mediante una petición HTTP que incluye un parámetro con el

nombre del archivo de imagen, lo cual constituye un punto de entrada potencialmente vulnerable a manipulación.

5. Fase 2: Interceptación de la petición de imagen

Con el proxy de Burp Suite activo, se navegó por el catálogo de productos mientras se registraba el tráfico HTTP generado. En el historial de peticiones (HTTP history) se identificaron múltiples solicitudes GET con el siguiente formato:

```
GET /image?filename=45.jpg HTTP/2
```

Este patrón se repetía para cada imagen del catálogo, cambiando únicamente el valor numérico del parámetro filename, lo que confirmó que dicho parámetro es utilizado directamente por el servidor para localizar y devolver un archivo del sistema.

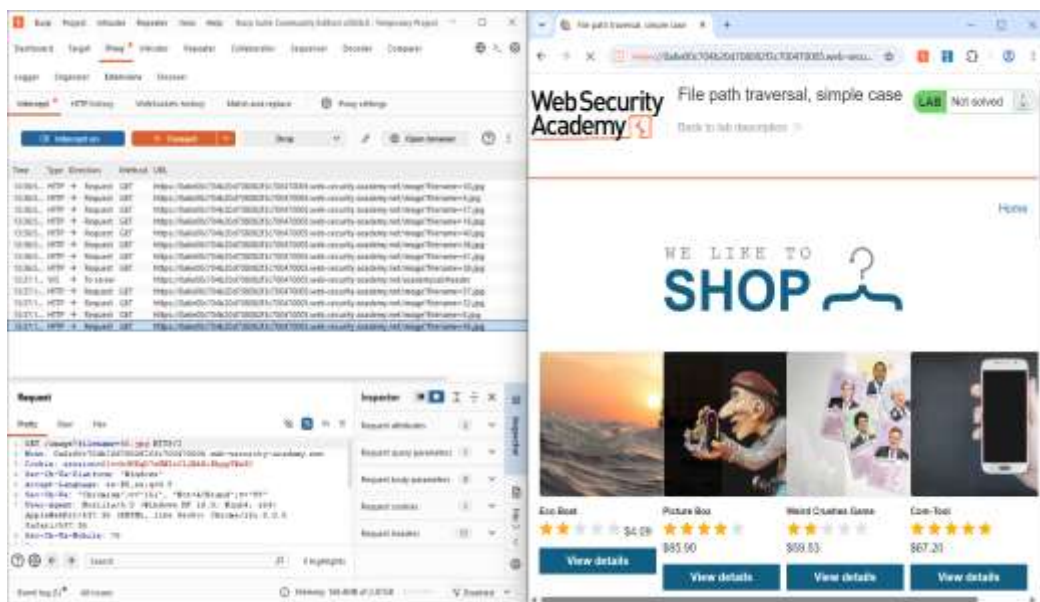


Figura 7. Historial de peticiones en Burp Proxy mostrando múltiples solicitudes GET /image?filename=N.jpg.

Se seleccionó una de estas peticiones y, mediante el menú, se envió a la herramienta Repeater para poder modificarla y reenviarla manualmente.

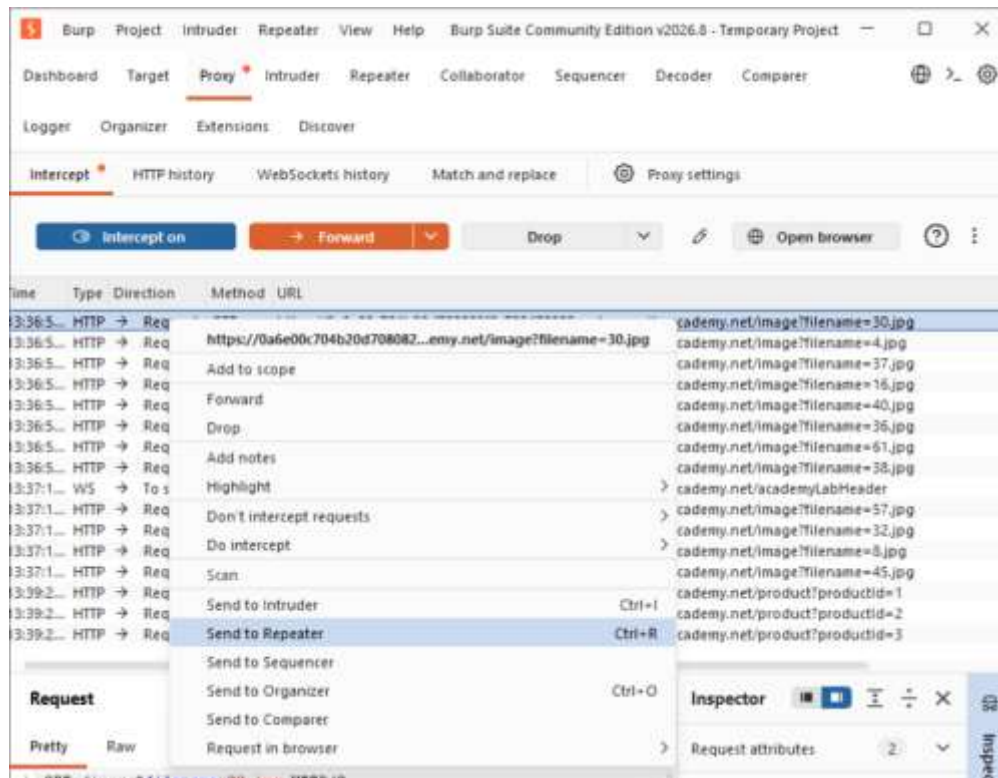


Figura 8. Envío de la petición seleccionada (filename=30.jpg) a Burp Repeater mediante “Send to Repeater”.

Concepto teórico: Burp Repeater. Repeater es el módulo de Burp Suite que permite tomar una petición HTTP capturada, editarla libremente como parámetros, cabeceras, método, cuerpo, etc. Y reenviarla al servidor de forma controlada, observando la respuesta en tiempo real. Es la herramienta adecuada para probar manualmente hipótesis de explotación, como la manipulación de un parámetro de nombre de archivo, sin necesidad de repetir la interacción completa desde el navegador.

6. Fase 3: Confirmación del parámetro vulnerable

Dentro de Repeater se inspeccionó la petición original, confirmando que el valor del parámetro filename correspondía exactamente al nombre del archivo de imagen solicitado, en este caso 30.jpg, sin ningún tipo de codificación adicional ni token de validación asociado.

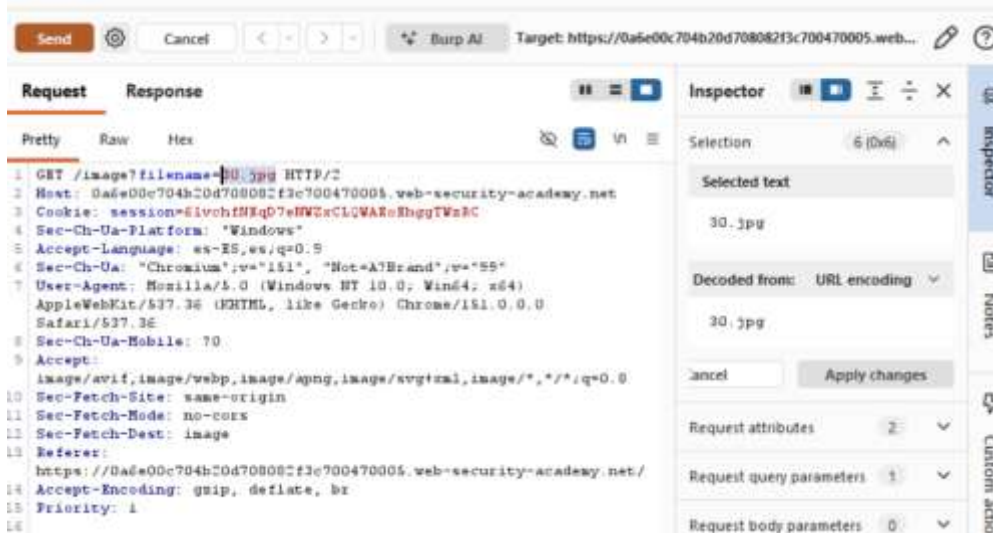


Figura 9. Petición original en Repeater: GET /image?filename=30.jpg, con el valor del parámetro resaltado en el Inspector.

Esta observación es la base de la hipótesis de explotación: si la aplicación concatena directamente el valor de filename a una ruta base en el servidor (por ejemplo, /var/www/images/ + filename) sin validar su contenido, sería posible manipular dicho valor para “escapar” del directorio de imágenes y acceder a otras rutas del sistema de archivos.

7. Fase 4: Explotación: construcción del payload

Se modificó el valor del parámetro filename en Repeater, reemplazándolo por la siguiente cadena:

```
GET /image?filename=../../../../etc/passwd HTTP/2
```

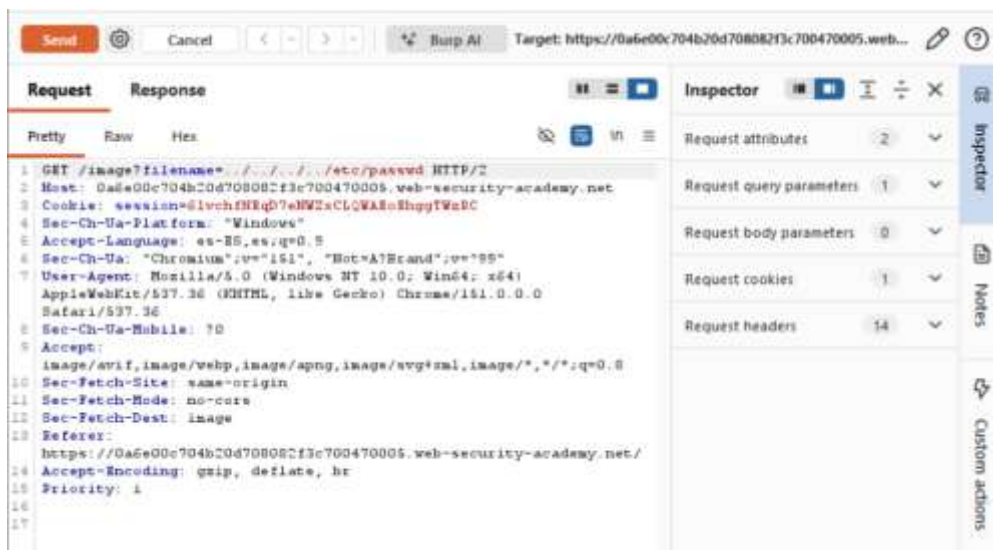


Figura 10. Petición modificada en Repeater con el payload de path traversal `../../../../etc/passwd`.

Al enviar la petición modificada, el servidor respondió con un código HTTP 200 OK y, en el cuerpo de la respuesta, con el contenido íntegro del archivo `/etc/passwd` del sistema operativo del servidor, confirmando la explotación exitosa de la vulnerabilidad.

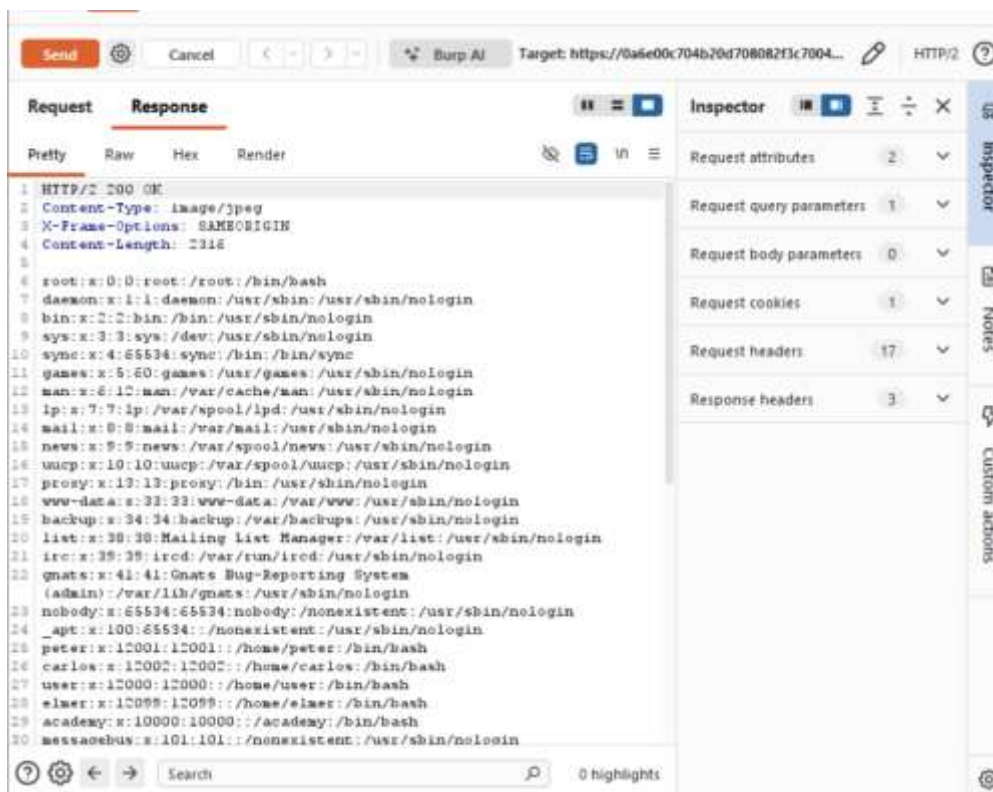


Figura 11. Respuesta del servidor mostrando el contenido completo del archivo `/etc/passwd`.

7.1 Explicación del payload

El payload se construye a partir de dos componentes:

- `../../../../` : cuatro secuencias de retroceso de directorio. Cada “`../`” instruye al sistema operativo para subir un nivel en la jerarquía de carpetas respecto al directorio base donde la aplicación busca las imágenes (por ejemplo, algo similar a `/var/www/images/`). Se utilizan varias secuencias encadenadas para garantizar que, sin importar la profundidad real del directorio de imágenes, el recorrido llegue hasta la raíz del sistema de archivos (`/`).
- `etc/passwd`: ruta absoluta, ya ubicada en la raíz, hacia el archivo estándar de Unix/Linux que contiene el listado de cuentas de usuario del sistema (nombre de usuario, UID, GID, directorio home y shell asignada). Aunque este archivo no almacena las contraseñas en texto plano, es un objetivo clásico para demostrar path traversal porque es legible por cualquier usuario del sistema y su contenido es fácil de reconocer.

Al concatenar el payload con la ruta base que utiliza la aplicación, el resultado final resuelto por el sistema operativo equivale a `/etc/passwd`, fuera del directorio que la aplicación pretendía exponer.

Concepto teórico: path traversal (directory traversal). Esta vulnerabilidad, clasificada dentro de la categoría A01:2021 – Broken Access Control del OWASP Top 10, ocurre cuando una aplicación utiliza entrada proporcionada por el usuario para construir una ruta de acceso a archivos en el sistema de archivos del servidor, sin validar ni sanitizar adecuadamente dicha entrada. Esto permite a un atacante manipular la ruta para acceder a archivos y directorios fuera del alcance previsto, incluyendo archivos de configuración, código fuente o archivos sensibles del sistema operativo.

7.2 Resolución del laboratorio

Al reenviar la petición modificada y comprobar que el servidor devolvió el contenido de `/etc/passwd`, PortSwigger marcó automáticamente el laboratorio como resuelto, confirmando el cumplimiento del objetivo planteado.



Figura 12. Confirmación de PortSwigger: “Congratulations, you solved the lab!”

8. Mitigación recomendada

Para prevenir vulnerabilidades de path traversal como la explotada en este laboratorio, se recomienda aplicar las siguientes medidas, de forma combinada:

- Evitar recibir nombres o rutas de archivo directamente del usuario. Sustituir el parámetro filename por un identificador indirecto que el servidor resuelva internamente contra una lista o base de datos de archivos permitidos.
- Validar y depurar (sanitizar) la entrada: rechazar cualquier valor que contenga secuencias “../”, “..\\”, su codificación URL (%2e%2e%2f) o caracteres de ruta absoluta, aceptando únicamente los caracteres esperados para un nombre de archivo.
- Ejecutar el servicio con el mínimo privilegio necesario y, cuando sea posible, aislar el proceso mediante chroot jail o contenedores, de forma que, aunque ocurra un escape de directorio, no existan archivos sensibles accesibles.
- Utilizar las funciones o librerías seguras que ofrezca el framework o lenguaje para servir archivos estáticos, en lugar de construir rutas manualmente mediante concatenación de cadenas.
- Reforzar la defensa en profundidad con un WAF (Web Application Firewall) que detecte y bloquee patrones de path traversal, y aplicar monitoreo/alertas sobre accesos a archivos fuera de lo esperado.
- Restringir los permisos de lectura de archivos sensibles del sistema (como /etc/shadow) y mantener actualizado el sistema operativo y el software del servidor web.

9. Conclusiones

Este laboratorio permitió demostrar de forma práctica cómo una validación insuficiente de la entrada del usuario en un parámetro que construye rutas de archivo puede derivar en una vulnerabilidad crítica de exposición de información. A partir de la simple observación del patrón de una petición HTTP (filename=N.jpg), fue posible formular y confirmar una hipótesis de explotación utilizando únicamente Burp Suite Community Edition, sin necesidad de herramientas de explotación automatizadas.

El ejercicio reforzó conceptos fundamentales de la metodología de pruebas de penetración web: identificación de la superficie de ataque, interceptación y análisis de tráfico HTTP y construcción manual de un payload de explotación, todo ello dentro de un entorno de laboratorio controlado y autorizado.